

DevOps Internship Task

Fix, test and explain the supplied environment.

You receive a Flask app, broken environment and three logs. Hidden issue types and count are not disclosed. Verify starter values.

Tools: Linux/WSL, Git, Docker, Compose, NGINX, PostgreSQL, Redis, Bash/Python, GitHub Actions.

Started Pack:

Gitlab Repo: [bargsystems / BARQ-Academy · GitLab](#)

Access Token: glpat-JguXYwIBW0rQTGCEGNU0IGM6MQpvOjEKdTptZnRpNA8.01.171s5od37

Part 1: Investigate

- Create your GitHub repository. Keep the baseline; commit before technical changes.
- Commit progressively: investigate -> fix -> verify. Use meaningful messages, not bulk uploads.
- For each issue, record symptoms, hypotheses, commands, results and failed attempts.
- Add the root cause, fix, retest evidence and commit. Claim fixes only when proven.
- Analyze all three logs with Bash/Python/Linux tools. Keep originals unchanged.
- Correlate access, error and app logs. Answer every log-template question.
- Show reproducible commands, counts, a timeline and conclusions; e.g. errors by time.

Part 2: Docker, networking and NGINX

- Use Dockerfile, docker-compose.yml, app/ and nginx/nginx.conf.
- Run two Flask instances behind NGINX, with working PostgreSQL and Redis connections.
- Publish only NGINX on host port 8080. Do not publish app, PostgreSQL or Redis ports.
- Connect NGINX + apps to frontend; apps + PostgreSQL + Redis to backend.
- Block direct NGINX access to PostgreSQL/Redis. Use service names, not container IPs.
- Before the video, name containers app-01, app-02, nginx, postgres and redis.
- Keep network names ending in frontend and backend. Return distinct app identities.
- Use a named PostgreSQL volume. Configure Redis persistence where appropriate.
- Set correct environment variables, health/readiness checks, restart policies and resource limits.
- Use required dependencies only. Avoid root/privileged operation where practical.
- Use health-check tools installed in the image. Explain base-image and health-check choices.
- Keep secrets out of images, code and Compose. Ignore secret files; provide a safe .env.example.

Required endpoints

- /: app response. /health: process liveness. /ready: PostgreSQL + Redis readiness.
- /instance: backend identity. /records: create/list PostgreSQL records.
- /counter: Redis-backed counter. Use real database/cache operations.

Part 3: Validation, persistence and CI

- Write validate.sh or validate.py. Use bounded waits, PASS/FAIL and non-zero failure exits.
- Check public access, all endpoints, both backends and PostgreSQL/Redis readiness.
- Check network isolation and prohibited host ports.
- Write failure_test.sh/.py: stop one backend, check availability, restore it and verify recovery.
- Measure traffic and errors during failure. Prove the recovered backend serves requests.
- Write backup.sh and restore.sh (or equivalents). Prove a PostgreSQL backup restores.
- Create a record through /records. Recreate app and PostgreSQL containers, keeping the volume.
- Prove the record survives. Document exact test, backup and restore commands.
- Add .github/workflows/ci.yml. Run on push and pull request.
- CI: checkout -> syntax/Compose checks -> build -> start -> wait for readiness -> validate.
- Fail CI when validation fails. Link the run for your final commit.
- Extra credit (optional): add and document an image/security scan.

Part 4: Documentation

- README.md: copyable setup, build, start/stop, test, failure, backup/restore and cleanup.
- troubleshooting.md: investigation journal, including failed attempts and retests.
- log_analysis.md: all template answers, commands, counts and correlated evidence.
- decisions.md: at least 5 decisions, assumptions, alternatives, trade-offs and limitations.
- security_review.md: at least 8 concrete risks/improvements in your solution.
- Cover secrets, ports, container user, images, networks, backup, monitoring and availability.
- Separate implemented fixes from production plans. Include persistence and logging risks.
- AI_USAGE.md: tools, purpose, affected files and verification; or write None.
- architecture.png/.pdf: request flow, ports, networks, storage and health relationships.

Questions - answer in your README or reports

- What failed first? What proved the cause? Which failed attempt taught you something?
- What patterns did the logs reveal? How did you avoid double-counting requests?
- How do requests flow? Why these ports, networks and readiness checks?
- Why these timeouts, retries, restart settings and resource limits?
- When should validation fail? What does green CI prove, or not prove?
- Which single points of failure remain? How would you fix them in production?
- What would you improve? How did you verify AI-assisted work?

Part 5: Video demonstration - 12-18 minutes

Record one continuous screen video with live narration and real terminal commands.

No cuts, pauses, edits, speed-up, overlays, voice replacement or prerecorded demos. Face camera is optional.

- Show your repository, starting commit and clean git status.
- Build/start the stopped environment; show service health. Prebuilt images are allowed.
- Test `/`, `/health`, `/ready`, `/records` and `/counter`.
- Use `/instance` to prove both backends serve repeated requests through NGINX.
- Stop one backend. Show continued traffic and errors; recover it and prove it serves again.
- Show a created record surviving app and PostgreSQL container recreation.
- Run validation and the failure test. Demonstrate one historical-log finding.
- Run `./video_challenge.sh` once, for the first time in this video working copy.
- Diagnose and fix its runtime fault. Do not reset it with docker compose down.
- Change public port 8080 to 8090 live. Prove NGINX works on 8090.
- Add a third app instance live. Prove all three respond; rerun validation.
- Run git status and git diff. Explain changes, commit on screen and show commit hashes.
- Push video commits. Link each change to its commit and video timestamp.

Submit

- GitHub repository URL, final commit hash, matching CI run and accessible video URL.
- Include all scripts, README, diagram, reports, decision log and AI disclosure.
- Add an evidence index: requirement -> file/output -> commit -> video timestamp.
- Match GitHub, video and documents to the final three-instance setup on port 8090.
- Explain any later documentation-only commits.

rules

- We score understanding and ownership through GitHub, video and documentation.
- We review commit diffs, timing and progress, not just commit counts or working results.
- Hard fails: missing/inaccessible evidence, non-compliant video or skipped live actions.
- Hard fails: fabricated/unverifiable evidence or material contradictions between submissions.
- Honest technical failures or incomplete work can earn partial credit. Show your attempts.
- AI and external resources are allowed. Understand and verify everything you submit.
- Disclose AI use. We do not use AI detectors; AI use or timing alone is not proof of misconduct.
- Do not fabricate commit dates. Use synthetic lab data only; never submit real secrets.